`Matrix m n α` — Lean 4 / Mathlib Cheatsheet

Core namespace: `Matrix` Source: `Mathlib.Data.Matrix.Basic` + `Mathlib.LinearAlgebra.Matrix` (Mathlib4)

> `Matrix m n α` is defined as `m → n → α`. Rows are indexed by `m`, columns by `n` (typically `Fin r` and `Fin c`). Element access: `A i j`. Most lemmas require `Fintype` instances on `m` and `n`, and algebraic instances (`CommRing`, `Field`, …) on `α`. The `![]` literal notation requires `import Mathlib.Data.Matrix.Basic`.

Type & Notation

`Matrix m n α` — m×n matrix with entries in α

```
def Matrix (m n α : Type*) := m → n → α
```

`A i j` — element at row i, column j

```
(A : Matrix m n α) → m → n → α
```

`Aᵀ` — transpose (swap rows/cols)

```
Matrix.transpose A : Matrix n m α
```

`A *ᵥ v` — matrix–vector product

```
Matrix.mulVec A v : m → α
```

`v ᵥ* A` — vector–matrix product

```
Matrix.vecMul v A : n → α
```

`u ⬝ᵥ v` — dot product of two vectors (= ∑ i, u i * v i)

```
Matrix.dotProduct u v : α
```

Constructors & Special Matrices

`Matrix.of f` — build from function (defeq but cleaner)

```
Matrix.of : (m → n → α) → Matrix m n α
```

`![![a, b], ![c, d]]` — literal for Fin-indexed matrices

```
notation via Matrix.cons / Matrix.empty
```

`(0 : Matrix m n α)` — zero matrix (all entries 0)

```
∀ i j, (0 : Matrix m n α) i j = 0
```

`(1 : Matrix n n α)` — identity matrix

```
1 i j = if i = j then 1 else 0
```

`Matrix.diagonal d` — diagonal from vector d; off-diag = 0

```
Matrix.diagonal : (n → α) → Matrix n n α
```

`Matrix.diag A` — extract main diagonal as a vector

```
Matrix.diag : Matrix n n α → n → α
```

`Matrix.stdBasisMatrix i j a` — standard basis matrix $e_{ij} \cdot a$

```
(i,j)-entry = a, all others = 0
```

`Matrix.fromBlocks A B C D` — assemble [[A, B], [C, D]]

```
Matrix.fromBlocks : block 2×2 matrix
```

Reindexing & Slicing

`Matrix.submatrix A r c` — reindex rows/cols (select a sub-block)

```
Matrix.submatrix : (m' → m) → (n' → n) → Matrix m' n' α
```

`Matrix.updateRow A i v` — replace row i with vector v

```
Matrix.updateRow : Matrix m n α → m → (n → α) → Matrix m n α
```

`Matrix.updateColumn A j v` — replace column j with vector v

```
Matrix.updateColumn : Matrix m n α → n → (m → α) → Matrix m n α
```

Arithmetic

`A + B` / `A - B` — entry-wise addition / subtraction

```
pointwise via Pi.instAdd
```

`A * B` — matrix multiplication (ring composition)

```
Matrix.mul : (A * B) i k = ∑ j, A i j * B j k
```

`r • A` — scalar multiplication

```
r • A  : every entry scaled by r
```

`Matrix.mul_assoc` — associativity

```
A * B * C = A * (B * C)
```

`Matrix.mul_add` / `Matrix.add_mul` — distributivity (left / right)

```
A * (B + C) = A*B + A*C  /  (A+B)*C = …
```

`Matrix.one_mul` / `Matrix.mul_one` — identity laws

`1 * A = A  /  A * 1 = A`

`Matrix.zero_mul  /  Matrix.mul_zero` annihilation

`0 * A = 0  /  A * 0 = 0`

Transpose

`Matrix.transpose_transpose` involution

`(Aᵀ)ᵀ = A`

`Matrix.transpose_mul` reversal law

`(A * B)ᵀ = Bᵀ * Aᵀ`

`Matrix.transpose_add` additive linearity

`(A + B)ᵀ = Aᵀ + Bᵀ`

`Matrix.transpose_smul` scalar commutes with ᵀ

`(r • A)ᵀ = r • Aᵀ`

`Matrix.transpose_zero  /  .transpose_one` zero and one are fixed points

`(0 : Matrix m n α)ᵀ = 0  /  (1)ᵀ = 1`

`Matrix.transpose_diagonal` diagonal matrices are symmetric

`(diagonal d)ᵀ = diagonal d`

`Matrix.transpose_apply` unfold for simp / calc proofs

`Aᵀ i j = A j i`

Trace

`Matrix.trace A` sum of diagonal entries

`Matrix.trace A = ∑ i, A i i`

`Matrix.trace_add` additive

`trace (A + B) = trace A + trace B`

`Matrix.trace_smul` scalar factor

`trace (r • A) = r * trace A`

`Matrix.trace_transpose` transpose-invariant

`trace Aᵀ = trace A`

`Matrix.trace_mul_comm` cyclic property

`trace (A * B) = trace (B * A)`

`Matrix.trace_zero  /  .trace_one` constants

`trace 0 = 0  /  trace 1 = ↑(Fintype.card n)`

Determinant

`Matrix.det A` Leibniz / permutation formula

`det A = ∑ σ : Equiv.Perm n, σ.sign • ∏ i, A i (σ i)`

`Matrix.det_mul` multiplicativity

`det (A * B) = det A * det B`

`Matrix.det_transpose` transpose-invariant

`det Aᵀ = det A`

`Matrix.det_one` identity

`det (1 : Matrix n n α) = 1`

`Matrix.det_zero` zero matrix

`det (0 : Matrix n n α) = 0  [Nonempty n]`

`Matrix.det_diagonal` product of diagonal entries

`det (diagonal d) = ∏ i, d i`

`Matrix.det_fin_two` explicit 2×2 formula

`det ![![a,b],![c,d]] = a*d - b*c`

`Matrix.det_smul` scalar factor

`det (r • A) = r ^ Fintype.card n * det A`

Inverse

`A⁻¹` inverse; defined on all square matrices

`Matrix.nonsing_inv A  (= 0 when det A = 0)`

`Matrix.nonsing_inv_mul` left inverse (nonsingular)

`A.det ∈ nonZeroDivisors α → A⁻¹ * A = 1`

`Matrix.mul_nonsing_inv` right inverse (nonsingular)

`A.det ∈ nonZeroDivisors α → A * A⁻¹ = 1`

`Matrix.nonsing_inv_nonsing_inv` double inverse

`IsUnit A.det → (A⁻¹)⁻¹ = A`

`Matrix.det_nonsing_inv` det of inverse

`det A⁻¹ = (det A)⁻¹`

`Matrix.isUnit_iff_isUnit_det` invertibility ↔ det is a unit

`IsUnit A ↔ IsUnit A.det`

mulVec & dotProduct

`Matrix.mulVec_add` linearity in the vector

```
A *ᵥ (u + v) = A *ᵥ u + A *ᵥ v
```

`Matrix.mulVec_smul` — scalar commutes

```
A *ᵥ (r • v) = r • A *ᵥ v
```

`Matrix.add_mulVec` — linearity in the matrix

```
(A + B) *ᵥ v = A *ᵥ v + B *ᵥ v
```

`Matrix.mul_mulVec` — associativity with $*_v$

```
(A * B) *ᵥ v = A *ᵥ (B *ᵥ v)
```

`Matrix.dotProduct_mulVec` — adjoint / transpose identity

```
u ⬝ᵥ (A *ᵥ v) = (Aᵀ *ᵥ u) ⬝ᵥ v
```

`Matrix.mulVec_one` / `Matrix.one_mulVec` — identity matrix as $*_v$

```
1 *ᵥ v = v  /  A *ᵥ (1-vec) = A-row-sums
```

Common proof strategies for `Matrix` goals:

- `ext i j` — reduce `A = B` to `∀ i j, A i j = B i j`
- `simp [Matrix.mul_apply]` — unfold matrix product entry
- `simp [Matrix.transpose_apply]` — unfold `Aᵀ i j = A j i`
- `simp [Matrix.diagonal_apply]` — unfold diagonal entry
- `ring` / `linarith` — close arithmetic goals after unfolding
- `simp [Matrix.det_fin_two]` — evaluate 2×2 determinant
- `rw [Matrix.trace_mul_comm]` — swap order under trace
- `simp [Finset.sum_add_distrib]` — split finite sums
- `norm_num` — decide concrete `Fin`-indexed goals
- `decide` — small fully concrete cases (e.g. `Matrix (Fin 2) (Fin 2) Bool`)